অধ্যায় ১৮

Functional Programming, Immutability এবং Advanced Data Transformation

Level: Intermediate → Advanced | **Project:** SalesFlow Analytics | **Prerequisites:** Functions, Arrays, Objects, Modules।

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি Pure Function ও Side Effect আলাদা করতে, Immutable Transformation লিখতে, `map` / `filter` / `reduce`-এর মাধ্যমে Data Pipeline তৈরি করতে, Grouping ও Ranking করতে, Function Composition বুঝতে এবং Automated Test দিয়ে Data Integrity যাচাই করতে পারবে।

১৮.১ Functional Programming কী এবং কেন?

Functional Programming (FP) হলো এমন একটি Programming Style যেখানে সমস্যাকে ছোট, পুনর্ব্যবহারযোগ্য Function-এর মাধ্যমে সমাধান করা হয়। বিশেষ গুরুত্ব দেওয়া হয় **Pure Functions**, **Immutable Data** এবং **Declarative Transformation**-এ। JavaScript Multi-Paradigm Language: একই Application-এ FP, OOP এবং Imperative Code একসঙ্গে থাকতে পারে।

ধরা যাক একটি Sales Dashboard-এর জন্য Orders থেকে Paid Revenue বের করতে হবে। Imperative পদ্ধতিতে Loop, Temporary Variable এবং State Update করতে পারো। Functional পদ্ধতিতে Transformation-এর উদ্দেশ্য সরাসরি প্রকাশ করা যায়।

```javascript
const orders = [
  { id: 1, status: 'paid', total: 800 },
  { id: 2, status: 'refunded', total: 500 },
  { id: 3, status: 'paid', total: 1200 }
];

const revenue = orders
  .filter(order => order.status === 'paid')
  .reduce((sum, order) => sum + order.total, 0);
console.log(revenue); // 2000
```

এখানে Original `orders` Array বদলায়নি। প্রতিটি Step-এর দায়িত্বও আলাদা।

১৮.২ Pure Function, Referential Transparency ও Side Effects

একটি Function-কে Pure বলা যায় যদি (১) একই Input দিলে একই Output দেয় এবং (২) External State পরিবর্তন না করে।

```javascript
function calculateLineTotal(quantity, unitPrice) {
  return quantity * unitPrice;
}
```

এটি Pure। কিন্তু নিচের Function বাইরের `total` পরিবর্তন করে:

```javascript
let total = 0;
function addToTotal(amount) {
  total += amount; // side effect
  return total;
}
```

Side Effect সবসময় খারাপ নয়। Browser DOM Update, Network Request, File Write এবং Logging বাস্তব Application-এর প্রয়োজনীয় কাজ। ভালো Architecture-এ এসবকে Core Calculation থেকে আলাদা রাখা হয়। এতে Test সহজ হয়।

Referential Transparency: Pure Function Call-কে একই Input-এর Result দিয়ে প্রতিস্থাপন করলেও আচরণ বদলায় না। এটি Reasoning ও Caching সহজ করে।

১৮.৩ Immutability — পরিবর্তনের বদলে নতুন Value

JavaScript-এ Object ও Array Reference-based। `const` দিয়ে Variable Reassignment বন্ধ করা যায়; Nested Data Mutation বন্ধ হয় না।

```javascript
const student = { name: 'Amina', profile: { city: 'Dhaka' } };
student.profile.city = 'Khulna'; // বৈধ, যদিও student const

const updated = {
  ...student,
  profile: { ...student.profile, city: 'Sylhet' }
};
```

Spread Operator একটি **Shallow Copy** তৈরি করে। Deeply Nested Structure-এর ক্ষেত্রে প্রয়োজনীয় স্তরগুলো Copy করতে হয়, অথবা Supported Environment-এ `structuredClone()` ব্যবহার করা যায়। তবে Function, DOM Node, কিছু Special Object `structuredClone`-এ Unsupported হতে পারে।

Array-এর জন্য:

```javascript
const items = ['A', 'B'];
const appended = [...items, 'C'];
const removed = items.filter(item => item !== 'A');
const replaced = items.map(item => item === 'B' ? 'New B' : item);
console.log(items); // ['A', 'B'] — unchanged
```

`sort()`, `reverse()`, `splice()` Original Array পরিবর্তন করে। Non-mutating বিকল্প হিসেবে `toSorted()`, `toReversed()`, `toSpliced()` Modern Runtimes-এ পাওয়া যায়; Compatibility পরীক্ষা করবে।

১৮.৪ `map()`, `filter()`, `reduce()` এবং `flatMap()`

`map` একেকটি Item থেকে নতুন Item তৈরি করে; `filter` শর্ত অনুযায়ী Item রাখে; `reduce` অনেক Item থেকে একটি Accumulator তৈরি করে; `flatMap` Mapping-এর পর এক স্তর Flatten করে।

```javascript
const products = [
  { name: 'Book', price: 400, stock: 3 },
  { name: 'Course', price: 1200, stock: 0 },
  { name: 'Pen', price: 30, stock: 10 }
];
const availableNames = products
  .filter(product => product.stock > 0)
  .map(product => product.name);
// ['Book', 'Pen']
const totalStockValue = products.reduce(
  (sum, product) => sum + product.price * product.stock, 0
); // 1500
```

`reduce()`-এ Initial Value দেওয়া গুরুত্বপূর্ণ। Empty Array-এ Initial Value ছাড়া `reduce()` Error দেয়।

১৮.৫ Data Validation ও Normalization

External Data বিশ্বাস করা যায় না। API, CSV বা Form থেকে আসা Numeric Value String হতে পারে। Pipeline-এর শুরুতে Input Normalize এবং Validate করো।

```javascript
function normalizeOrder(order) {
  const quantity = Number(order.quantity);
  const unitPrice = Number(order.unitPrice);
  if (!Number.isSafeInteger(quantity) || quantity <= 0) {
    throw new RangeError('Quantity must be a positive integer');
  }
  if (!Number.isFinite(unitPrice) || unitPrice < 0) {
    throw new RangeError('Price must be non-negative');
  }
  return { ...order, quantity, unitPrice, total: quantity * unitPrice };
}
```

Financial Precision Note: বাস্তব Payment System-এ Floating-Point Price সরাসরি ব্যবহারে Precision Issue হতে পারে। Currency Minor Units (যেমন পয়সা) বা নির্ভরযোগ্য Decimal Library ব্যবহার করা উত্তম। আমাদের Project শিক্ষামূলক।

১৮.৬ Grouping এবং Aggregation

Category অনুযায়ী Sales Group করার সময় Accumulator ব্যবহার করা যায়:

```javascript
function revenueByCategory(orders) {
  return orders
    .filter(order => order.status === 'paid')
    .reduce((groups, order) => ({
      ...groups,
      [order.category]: (groups[order.category] ?? 0) + order.total
    }), {});
}
```

এটি Immutable, কিন্তু প্রতিটি Iteration-এ পুরো Object Spread করলে বড় Dataset-এ Cost বাড়তে পারে। Performance-sensitive Code-এ একটি Function-এর **Local Mutable Accumulator** ব্যবহার করেও বাইরে থেকে Pure আচরণ বজায় রাখা যায়:

```javascript
function efficientRevenueByCategory(orders) {
  const groups = new Map();
  for (const order of orders) {
    if (order.status !== 'paid') continue;
    groups.set(order.category, (groups.get(order.category) ?? 0) + order.total);
  }
  return groups;
}
```

মূল কথা: FP মানেই প্রতিটি Local Variable অপরিবর্তনীয় হতে হবে, এমন নয়। External Observable Mutation এড়ানোই প্রধান উদ্দেশ্য।

১৮.৭ Function Composition ও Pipelines

একটি Function-এর Output আরেকটি Function-এর Input হলে Function Composition তৈরি হয়।

```javascript
const pipe = (...functions) => initial =>
  functions.reduce((value, fn) => fn(value), initial);

const trim = value => value.trim();
const lower = value => value.toLowerCase();
const hyphenate = value => value.replace(/\s+/g, '-');
const slugify = pipe(trim, lower, hyphenate);
console.log(slugify('  JavaScript Mastery  '));
```

`pipe` বাম থেকে ডানে কাজ করে। `compose` সাধারণত ডান থেকে বামে কাজ করে। Function Signatures মিলতে হবে।

১৮.৮ Closures, Currying ও Partial Application

```javascript
const greaterThan = minimum => number => number > minimum;
const premiumPrices = [100, 500, 1200].filter(greaterThan(400));
// [500, 1200]

const multiply = a => b => a * b;
const double = multiply(2);
console.log(double(7)); // 14
```

এখানে Inner Function বাইরের `minimum` বা `a` ধরে রাখে—এটি Closure। Currying একটি Multi-Argument Function-কে একাধিক Single-Argument Function-এর Chain হিসেবে প্রকাশ করে। Partial Application কিছু Argument আগেই Fix করে নতুন Function তৈরি করে।

১৮.৯ Sorting, Stability ও Non-Mutation

```javascript
const ranked = [...products].sort((a, b) => b.price - a.price);
```

Original `products` অক্ষত থাকে। String Sort-এ `localeCompare()` উপযোগী। Multiple Sort Keys থাকলে Tie-breaker যোগ করো। Input Array বড় হলে বারবার Sorting-এর Cost সাধারণত O(n log n)।

১৮.১০ Async Pipelines এবং সীমাবদ্ধতা

`map(async ...)` একটি **Promise Array** তৈরি করে, Resolved Value Array নয়।

```javascript
const ids = [1, 2, 3];
const results = await Promise.all(ids.map(async id => {
  return { id, valid: true };
}));
```

`Promise.all`-এ একটি Promise Reject করলে Combined Promise Reject হয়। Concurrent Request বেশি হলে Rate Limit/Memory বিবেচনা করে Bounded Concurrency দরকার হতে পারে।

১৮.১১ Practical Project — SalesFlow Analytics

Goal: Raw Sales Orders থেকে Searchable Dashboard, Paid Revenue, Unit Count ও Category Ranking তৈরি করা।

Folder Structure:

```
source-code/chapter-18/
├── index.html
├── style.css
├── sales-core.mjs
├── app.mjs
└── sales-core.test.mjs
```

Architecture

```
Raw Orders
    │
    ▼
normalize + validate
    │
    ▼
filter (search/category/status)
    │
    ├──► Paid Revenue / Units
    ├──► Category Aggregation + Ranking
    └──► DOM Rendering (textContent only)
```

`sales-core.mjs`-এ `normalizeOrder`, `prepareOrders`, `filterOrders`, `sumRevenue`, `totalUnits`, `groupByCategory`, `rankCategories`, `summarize` ও `pipe` রয়েছে। এগুলো DOM-নির্ভর নয়, তাই Node.js Test Runner দিয়ে Test করা যায়। `app.mjs`-এ Event Listener ও DOM Rendering আলাদা রাখা হয়েছে। Full Runnable Files Companion Source Code-এর `source-code/chapter-18/` Folder-এ রয়েছে।

Core Usage:

```javascript
import { sampleOrders, summarize } from './sales-core.mjs';
const report = summarize(sampleOrders, { category: 'Books', status: 'paid' });
console.log(report.revenue); // 2900
console.log(report.units);   // 3
```

চালানোর নিয়ম: Project Folder-এ Local HTTP Server চালিয়ে Browser-এ `index.html` খুলো। যেমন `python -m http.server 8000`, তারপর `http://localhost:8000`। `file://` দিয়ে ES Modules চালালে Browser Security Restrictions হতে পারে। Test: `node --test sales-core.test.mjs`।

Design Decisions: Source Array Mutate করা হয়নি; `textContent` ব্যবহার করা হয়েছে যাতে User Text HTML হিসেবে Execute না হয়; Refund/Pending Revenue-তে Count হয় না; Empty Dataset-এর জন্য Safe Default আছে।

১৮.১২ Testing Strategy

Pure Functions Test করতে Browser লাগে না। গুরুত্বপূর্ণ Cases:

1. Empty Array-তে Revenue 0।
2. Refunded Order Revenue-তে যোগ হবে না।
3. Duplicate Order ID Reject হবে।

4. Negative Price বা Zero Quantity Reject হবে।
5. Search Case-insensitive।
6. Category Ranking Descending।
7. Input Array Mutation হবে না।

Sample Test:

```javascript
import test from 'node:test';
import assert from 'node:assert/strict';
import { summarize } from './sales-core.mjs';

test('empty report', () => {
  assert.equal(summarize([]).revenue, 0);
});
```

১৮.১৩ Common Mistakes

- `map()` ব্যবহার করে শুধু Side Effect ঘটানো; এমন ক্ষেত্রে `forEach()` বা Loop বেশি স্পষ্ট।
- `sort()` দিয়ে Original Array অজান্তে বদলে ফেলা।
- `reduce()`-এ Initial Value বাদ দেওয়া।
- `const`-কে Deep Immutability মনে করা।
- `map(async ...)` থেকে Promise Array পাওয়ার বিষয়টি ভুলে যাওয়া।
- Monetary Values-এর Floating-Point Precision উপেক্ষা করা।
- Large Dataset-এ প্রতিটি `reduce` Iteration-এ Object Spread করে অপ্রয়োজনীয় Cost বাড়ানো।

১৮.১৪ Practice Exercises

১. `[1,2,3,4]` থেকে প্রতিটি সংখ্যার Square-এর Array তৈরি করো।
২. `products` থেকে Stock-এ থাকা Product Name বের করো।
৩. Paid Orders-এর মোট Revenue বের করো।
৪. Category অনুযায়ী Order Count তৈরি করো।
৫. Input Array না বদলে Price অনুযায়ী Descending Sort করো।
৬. `pipe()` দিয়ে Trim → Lowercase → Slug Generator বানাও।
৭. Duplicate ID Reject করার Function লিখো।
৮. Empty Dataset-এর জন্য Safe Summary তৈরি করো।
৯. `async map` এবং `Promise.all` দিয়ে Fake Data Enrichment করো।
১০. `groupByCategory()`-এর Unit Test লিখো।

১৮.১৫ Sample Solutions

```javascript
```

```
// 1
const squares = [1,2,3,4].map(n => n*n);
// 2
const inStock = products.filter(p => p.stock > 0).map(p => p.name);
// 3
const paidRevenue = orders.filter(o => o.status === 'paid')
  .reduce((sum,o) => sum + o.total, 0);
// 4
const countByCategory = orders.reduce((acc,o) => {
  acc[o.category] = (acc[o.category] ?? 0) + 1;
  return acc;
}, {});
// 5
const sorted = [...products].sort((a,b) => b.price - a.price);
// 6
const slug = pipe(s => s.trim(), s => s.toLowerCase(),
  s => s.replace(/\s+/g,'-'));
// 7
function rejectDuplicates(items) {
  const ids = items.map(item => item.id);
  if (new Set(ids).size !== ids.length) throw new Error('Duplicate IDs');
  return items;
}
// 8
const safeSummary = items => ({ count: items.length,
  total: items.reduce((sum,item) => sum + item.total,0) });
// 9
async function enrich(ids) {
  return Promise.all(ids.map(async id => ({id, valid:true})));
}
// 10: source-code/chapter-18/sales-core.test.mjs দেখো।
```

Exercise Note: Solutions 2–5-এ পূর্ববর্তী Sections-এর `products` ও `orders` Sample Variables ব্যবহার করা হয়েছে।

১৮.১৬ Knowledge Check — ১০টি MCQ

১. Pure Function-এর বৈশিষ্ট্য কোনটি? A) Global State Update B) Same Input → Same Output C) DOM Mutation D) Random Output

২. `filter()` কী Return করে? A) Boolean B) New Array C) Single Number D) Promise

৩. কোন Method Original Array Mutate করে? A) `map` B) `filter` C) `sort` D) `slice`

৪. `reduce([])`-এ Initial Value না থাকলে কী হয়? A) 0 B) null C) TypeError D) undefined

৫. `const obj = {x:1}` হলে `obj.x=2`? A) সবসময় Error B) সম্ভব C) Syntax Error D) Compile Error

৬. `map(async fn)` কী Return করে? A) Resolved Values B) Promise Array C) String D) Map Object

৭. `pipe(f,g)(x)` সাধারণত কোন Order? A) g তারপর f B) f তারপর g C) Random D) Parallel

৮. External State পরিবর্তন না করেও Local Mutation কি থাকতে পারে? A) না B) হ্যাঁ C) শুধু Browser-এ D) শুধু React-এ

৯. `structuredClone()` কি Function Clone করতে পারে? A) সবসময় B) না C) শুধু Async Function D) শুধু

Arrow Function

১০. Refund বাদ দিয়ে Revenue হিসাবের আগে কী প্রয়োজন? A) `filter` B) `alert` C) `document.write` D) `eval`

Answer Key ও Explanation

1. **B** — Deterministic Result ও Observable Side Effect না থাকা Pure Function-এর মূল বৈশিষ্ট্য।
2. **B** — `filter` নতুন Array দেয়।
3. **C** — `sort` In-place Mutate করে।
4. **C** — Empty Array-এ Initial Accumulator না থাকলে TypeError।
5. **B** — `const` Binding Reassignment আটকায়, Object Mutation নয়।
6. **B** — `async` Callback-এর Result Promise।
7. **B** — Pipe Left-to-Right।
8. **B** — Local Mutation বাইরের Observable State না বদলালে Function Pure থাকতে পারে।
9. **B** — Function `structuredClone`-এ Supported নয়।
10. **A** — Status Filter দিয়ে Eligible Orders নির্বাচন করা যায়।

১৮.১৭ Chapter Recap ও Next Step

Functional Programming একটি চিন্তার পদ্ধতি: ছোট Pure Functions, Immutable Data Flow, Explicit Validation এবং Testable Transformations। SalesFlow Analytics-এ আমরা এই ধারণাগুলোকে বাস্তব Project-এ ব্যবহার করেছি। পরবর্তী অধ্যায়ে Performance Optimization, Memory Management ও Web Application Efficiency নিয়ে কাজ করব।